EEP 467 Homework 2: Breaking CAPTCHAs End-to-End

In lab 2, we have successfully built a pipeline that automatically recognizes CAPTCHAs. So far, the recognition process requires manually identifying contours (and subsequently bounding boxes) around characters using expert rules, before performing classification on extracted character images. This inspires us for possible further improvement: can we use a single neural network to directly identify CAPTCHA characters from CAPTCHA image? In this homework, we will get rid of bounding box identification and build an end-to-end neural network model.

Before You Start

1. You can discuss on Discord if you have questions and want to seek help; however, please try your best to **limit the scope of your question** and **avoid asking directly for answers**. You should also **avoid copy-pasting answers and code** from others.
2. You are allowed to use AI assistants for help. In this case, you should **acknowledge your use of AI assistant** by providing its **name as well as how it helps you** to obtain your answer. Again, you **should not copy-paste from the response** of the AI assistant.

Pre-processing

First of all, make sure you have installed all dependencies needed for this homework. The list of depedencies is similar to lab 3, except that we will ditch OpenCV and `imutils`, and instead use PyTorch Vision (`torchvision`) for some of our preprocessing work:

In [19]:
```
%pip install matplotlib scikit-learn tqdm

# If using Anaconda / Miniconda, install PyTorch and PyTorch Vision with:
# %conda install conda-forge::pytorch-gpu conda-forge::torchvision # (CPU and GPU support)
# %conda install conda-forge::pytorch conda-forge::torchvision # (CPU-only support)

# If using Pip, install PyTorch and PyTorch Vision with:
%pip install torch torchvision
# When installing PyTorch with GPU support, refer to https://pytorch.org/get-started/locally/
# if your CUDA version differs from the default CUDA version of the current version PyTorch package.
```

```
^C
Note: you may need to restart the kernel to use updated packages.
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: matplotlib in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (3.10.8)
Requirement already satisfied: scikit-learn in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (1.7.2)
Requirement already satisfied: tqdm in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (4.67.3)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from m
atplotlib) (1.3.2)
Requirement already satisfied: cycler>=0.10 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from matpl
otlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from
matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from
matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from matplo
tlib) (2.2.6)
Requirement already satisfied: packaging>=20.0 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from ma
tplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from matplotl
ib) (12.0.0)
Requirement already satisfied: pyparsing>=3 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from matpl
otlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (fr
om matplotlib) (2.9.0.post0)
Requirement already satisfied: scipy>=1.8.0 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from sciki
t-learn) (1.15.3)
Requirement already satisfied: joblib>=1.2.0 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from scik
it-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (fr
om scikit-learn) (3.6.0)
Requirement already satisfied: colorama in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from tqdm) (0.
4.6)
Requirement already satisfied: six>=1.5 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from python-da
teutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: torch in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (2.10.0+cpu)
Requirement already satisfied: torchvision in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (0.25.0+cpu)
Requirement already satisfied: filelock in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from torch)
(3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\kelly\anaconda\envs\torch_env\lib\site-package
s (from torch) (4.15.0)
Requirement already satisfied: sympy>=1.13.3 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from torc
h) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from to
rch) (3.4.2)
Requirement already satisfied: jinja2 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from torch) (3.
1.6)
Requirement already satisfied: fsspec>=0.8.5 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from torc
h) (2025.12.0)
Requirement already satisfied: numpy in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from torchvision)
(2.2.6)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (f
rom torchvision) (12.0.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from
sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\kelly\anaconda\envs\torch_env\lib\site-packages (from ji
nja2->torch) (3.0.2)
Note: you may need to restart the kernel to use updated packages.
```

Once you are ready, we will start with data pre-processing step where we load all CAPTCHA images into memory. There are a few utilities in `torchvision` which can help us do the job:

In [20]:
```
import os

import torch
from torchvision.datasets.folder import IMG_EXTENSIONS
from torchvision.io import ImageReadMode, decode_image

# CAPTCHA images directory
CAPTCHA_IMAGES_DIR = "captcha-images"

# CAPTCHA images
captcha_images = []
# CAPTCHA texts
captcha_texts = []

## [ TODO ]
# 1) Load all CAPTCHA images from `CAPTCHA_IMAGES_DIR` into `captcha_images`
#    (Hint: use `os.scandir` to iterate over directory entries and check file extensions with `IMG_EXTENSIONS`)
#    (Hint: use `torchvision.io.decode_image` to load images in grayscale mode)
#    (Note: your code should ignore non-image files)
# 2) Stack all CAPTCHA images into a PyTorch tensor
# 3) Load all CAPTCHA texts (filenames without extension) into `captcha_texts`
#    (Items in `captcha_images` and `captcha_texts` should match)
with os.scandir(CAPTCHA_IMAGES_DIR) as files:
    for file in files:
        if file.is_file() and file.name.lower().endswith(IMG_EXTENSIONS):
            captcha_images.append(decode_image(file.path, mode=ImageReadMode.GRAY))
            captcha_texts.append(os.path.splitext(file.name)[0])

captcha_images = torch.stack(captcha_images, dim =0)
assert isinstance(captcha_images, torch.Tensor), "`captcha_images` must be a PyTorch tensor!"
assert captcha_images.ndim==4, "`captcha_images` should be a 4D tensor!"
assert captcha_images.shape[1:]==(1, 24, 72), "`captcha_images` should have shape (n_images, 1, h, w)!"

assert len(captcha_images)==len(captcha_texts), \
    "`captcha_images` should have the same number of elements as `captcha_texts`!"
assert all(isinstance(text, str) and len(text)==4 for text in captcha_texts), \
    "all `captcha_texts` should be strings with four letters!"
```

Then, we split all CAPTCHA images and texts into training-validation and test sets:

In [21]:
```
from sklearn.model_selection import train_test_split

# Training-validation-test split seed
TVT_SPLIT_SEED = 31528476

# Split dataset into training and test sets
captcha_images_tv, captcha_images_test, captcha_texts_tv, captcha_texts_test = train_test_split(
    captcha_images, captcha_texts, test_size=0.2, random_state=TVT_SPLIT_SEED
)
```

Next, we encode each character in the CAPTCHA text as class indices:

In [22]:
```
from sklearn.preprocessing import LabelEncoder
import hw2_util

# Label encoder
le = LabelEncoder()

## [ TODO ]
# 1) Flatten training-validation CAPTCHA texts into list of characters
#    (Hint: `hw2_util.flatten` can help you with that)
# 2) Fit and transform characters into class indices using `LabelEncoder`
# 3) Save number of classes (distinct characters) in `n_classes`
# 4) Convert class indices to PyTorch tensor and reshape as `texts_class_indices_tv`
#    (The shape of `texts_class_indices_tv` should be (n_images, 4))
char_list = list(hw2_util.flatten(captcha_texts_tv))
class_idxs = le.fit_transform(char_list)
n_classes = len(le.classes_)
texts_class_indices_tv = torch.tensor(class_idxs)
texts_class_indices_tv = texts_class_indices_tv.reshape(len(captcha_texts_tv), 4)


assert isinstance(texts_class_indices_tv, torch.Tensor), "`texts_class_indices_tv` must be a PyTorch tensor!"
assert texts_class_indices_tv.ndim==2, "`texts_class_indices_tv` must have two dimensions!"
assert texts_class_indices_tv.shape==(len(captcha_texts_tv), 4), \
    "Shape of `texts_class_indices_tv` should be (n_images, 4)!"
```

Then, we further split training-validation set into two parts:

In [23]:
```
# Split training set further into training and validation sets
captcha_images_train, captcha_images_vali, texts_class_indices_train, texts_class_indices_vali = train_test_split(
    captcha_images_tv, texts_class_indices_tv, test_size=0.25, random_state=TVT_SPLIT_SEED
)
```

Data Augmentation

The following step will be unique to this homework: we will perform **data augmentation** using PyTorch's `transforms.Compose` and `transforms.RandomAffine`. Here we will configure the `RandomAffine` transform to **apply the following random effects** to our CAPTCHAs:

- Random rotation (at most 5 degrees)
- Random horizontal and vertical shifting (at most 10%)
- Random shearing (at most 5 degrees)
- Random zooming in or out (at most 10%)

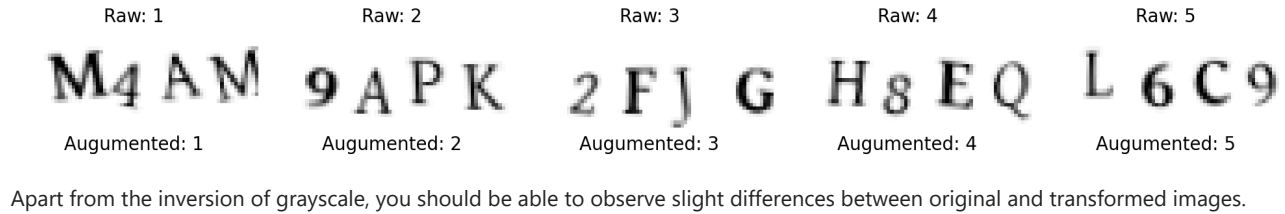
Besides, we will also **pre-process the image pixels** in the `RescaleInvertPixels` custom transform. Here we need to first **rescale the grayscale values** to $[0, 1]$ range and then **invert the grayscales**. We will chain the `RescaleInvertPixels` transform together with the `RandomAffine` transform using `Compose` to create a complete augmentation pipeline.

Below code will create such a transform pipeline, and preview its effects on a few sample CAPTCHAs:

In [24]:
```
from torch import nn
from torchvision.transforms import Compose, InterpolationMode, RandomAffine
from sklearn.preprocessing import MinMaxScaler

class RescaleInvertPixels(nn.Module):
    def forward(self, raw_image):
        ## [ TODO ]
        # 1) Rescale pixel grayscale values to [0, 1]
        # 2) Invert the grayscales of the image (darker becomes lighter, and vice versa)
        # 3) Return modified grayscales
        image = raw_image.float() / 255.0
        image = 1.0 - image
        return image



## [ TODO ]
# Create a transform pipeline using `Compose` with:
# 1) `RescaleInvertPixels()` for preprocessing
# 2) `RandomAffine` with the following random effects:
#    - Rotation: at most 5 degrees
#    - Horizontal and vertical shift: at most 10%
#    - Zooming: at most 10% (scale between 0.9 and 1.1)
#    - Shearing: at most 5 degrees
#    - Use `InterpolationMode.BILINEAR` for interpolation
#
# Choose parameters such that CAPTCHAs are moderately distorted but still recognizable.
captcha_transform = Compose([
    RescaleInvertPixels(),
    RandomAffine(degrees=5, translate=(0.1,0.1), scale=(0.9, 1.1),
                shear=5, interpolation=InterpolationMode.BILINEAR)
])

# Select a few images for preview
preview_orig = captcha_images_train[:5]
# Generate a few transformed CAPTCHA images for preview
preview_trans = captcha_transform(preview_orig)

# Preview original and transformed CAPTCHAs
hw2_util.print_images(
    preview_orig.squeeze(-3),
    texts=[f"Raw: {i+1}" for i in range(5)]
)
hw2_util.print_images(
    preview_trans.squeeze(-3),
    texts=[f"Augumented: {i+1}" for i in range(5)]
)
```

```
Figure(2000x500)
Figure(2000x500)
```

M4AM
Raw: 1
9APK
Raw: 2
2FJG
Raw: 3
H8EQ
Raw: 4
L6C9
Raw: 5



Apart from the inversion of grayscale, you should be able to observe slight differences between original and transformed images. These transformed CAPTCHAs should still be easily recognizable, though.

Training

From now on, we will shift our focus to the end-to-end neural network model. We assume our neural network model takes a bunch of CAPTCHA images (known as a batch) as input and outputs logits for each of the four character positions. In other words, if we have `n_classes` distinct characters in all CAPTCHAs, the output of our model would be a PyTorch tensor of shape `(batch_size, 4, n_classes)`. From each position's logits, we choose the character with the highest logit value (using the `argmax` operator), and we define a correct prediction as the one where all four characters are correctly predicted.

Our first task is to implement custom loss and accuracy functions for our task, since PyTorch's built-in functions expect different output formats. Both `pred_logits` (Raw model outputs) and `actual_class_indices` (Ground truth class indices) are batch tensors of shape `(batch_size, 4, n_classes)` and `(batch_size, 4)` respectively. Here we use PyTorch operations to compute our custom loss and accuracy metric:

In [25]:
```
from torch.nn import functional as f

def multi_char_loss(pred_logits, actual_class_indices):
    """ Compute the mean cross entropy loss of multiple CAPTCHA characters recognition. """
    return f.cross_entropy(pred_logits.flatten(0, 1), actual_class_indices.flatten())

def multi_char_acc(pred_logits, actual_class_indices):
    """ Compute the accuracy of multiple CAPTCHA characters recognition. """
    # Compute predicted class indices (most likely class / largest logits)
    pred_class_indices = pred_logits.argmax(-1)
    # Compute prediction accuracy for ALL four characters in a CAPTCHA, then average across samples
    return (pred_class_indices==actual_class_indices).all(-1).float().mean()
```

Next, we will complete `build_model` function which is responsible for building the neural network model. The neural network starts with several "convolution blocks" as usual, each of which contains a convolution layer for feature extraction and a max pooling layer for dimensionality reduction. The latter part of the network is however different from lab 3 in that **we replace regular fully-connected layers (implemented with linear layers) with "zone-wise" fully-connected layers (implemented with convolution layers)**.

We also substitute the regular cross-entropy loss for our custom `multi_char_loss` and make use of our custom accuracy metric, `multi_char_acc`, due to the unique output format of our model.

Below is the structure of our network:

In [59]:
```
from hw2_util import Transpose

def build_model(n_classes):
    """ Build the neural network model for CAPTCHA classification. """
    return nn.Sequential(
        # [ TODO ]
        # Complete the model architecture according to the specifications shown above and below:
        #
        # First convolution block: (*, 1, 24, 72) -Conv2d-> (*, 10, 24, 72) -ReLU-> (*, 10, 24, 72) -MaxPool2d-> (*,
        # 1) Convolution layer: 1 input channel, 10 output channels, 5*5 kernel, padded to maintain same shape
        # 2) ReLU activation layer
        # 3) Max pooling layer: 2*2 kernel
        nn.Conv2d(1, 10, 5, stride=1, padding=2),
        nn.ReLU(),
        nn.MaxPool2d(2),
        # Second convolution block: (*, 10, 12, 36) -Conv2d-> (*, 40, 12, 36) -ReLU-> (*, 40, 12, 36) -MaxPool2d-> (
        nn.Conv2d(10, 40, 5, stride=1, padding=2),
        nn.MaxPool2d(2),

        # Third convolution block: (*, 40, 6, 18) -Conv2d-> (*, 100, 6, 18) -ReLU-> (*, 100, 6, 18) -MaxPool2d-> (*,
        nn.Conv2d(40, 100, 5, stride=1, padding=2),
        nn.ReLU(),
        nn.MaxPool2d(2),
        # First "zone-wise" fully-connected block: (*, 100, 3, 9) -Conv2d-> (*, 800, 1, 4) -ReLU-> (*, 800, 1, 4)
        # Convolution layer: 100 input channels, 800 output channels, 3*3 kernel, horizontal stride set to 2 to obt
        nn.Conv2d(100, 800, 3, padding=0, stride=(1,2)),
        # Second "zone-wise" fully-connected layer: (*, 800, 1, 4) -Conv2d-> (*, n_classes, 1, 4)
        nn.Conv2d(800, n_classes, 1),
        # Convolution layer: `n_classes` channels, 1*1 kernel, no padding, used for dimensionality reduction
        nn.Conv2d(n_classes, n_classes, 1),
        # Reshape and transpose dimensions: (*, n_classes, 1, 4) -Flatten-> (*, n_classes, 4) -`lab_3_helpers.Transp
        nn.Flatten(2),
        Transpose(1,2)

    )
```

To facilitate efficient data loading and transformation, we will create a custom PyTorch `Dataset` class called `CAPTCHADataset`. This class will handle loading CAPTCHA images and their corresponding labels, and apply the data augmentation transforms on-the-fly during training. By implementing the standard PyTorch `Dataset` interface, we can leverage PyTorch's `DataLoader` for efficient batch processing and parallel data loading:

In [27]:
```
from torch.utils.data import Dataset

class CAPTCHADataset(Dataset):
    def __init__(self, captcha_images, texts_features, captcha_transform):
        self.captcha_images = captcha_images
        self.texts_features = texts_features
        self.captcha_transform = captcha_transform

    def __len__(self):
        return len(self.captcha_images)

    def __getitem__(self, idx):
        captcha_image = self.captcha_transform(self.captcha_images[idx])
        text_features = self.texts_features[idx]

        return captcha_image, text_features
```

Finally, the `train_model` function implements the standard PyTorch training loop. For each epoch, it performs forward passes on batches from the training set, computes loss using `multi_char_loss`, backpropagates gradients, and updates model parameters. After each epoch, it evaluates the model on the validation set. Data augmentation is applied on-the-fly via the `CAPTCHADataset` during both training and validation.

In [65]:
```
from tqdm import tqdm
from torch.utils.data import DataLoader

def train_model(model, optimizer, dataset_train, dataset_vali, batch_size=32, n_epochs=50, device=None):
    # Detect device from model parameters if not provided
    device = device or next(iter(model.parameters())).device

    ## [ TODO ]
    # 1) Create training data loader from `dataset_train` with shuffling enabled
    # 2) Create validation data loader from `dataset_vali`
    # (Note: set the batch size for both loaders)
    loader_train = DataLoader(dataset_train, batch_size, shuffle=True)
    loader_vali = DataLoader(dataset_vali, batch_size, shuffle=True)

    history = []

    print("Starting training ...")
    for i in range(n_epochs):
        # In every epoch, do training first ...
        loss_train = 0.
        acc_train = 0.

        print(f"Starting training epoch {i+1}/{n_epochs} ...")
        for captcha_images_batch, texts_class_indices_batch in tqdm(loader_train):
            ## [ TODO ]
            # Complete the training inner loop:
            # 1) Move images and labels to the target device
            captcha_images_batch = captcha_images_batch.to(device)
            texts_class_indices_batch = texts_class_indices_batch.to(device)
            # 2) Perform forward pass to get logits
            output = model(captcha_images_batch)
            # 3) Compute loss using `multi_char_loss` and accuracy using `multi_char_acc`
            loss = multi_char_loss(output, texts_class_indices_batch)
            acc = multi_char_acc(output, texts_class_indices_batch)
            # 4) Perform backward propagation
            loss.backward()
            # 5) Update model parameters by taking an optimizer step
            optimizer.step()
            # 6) Clear model gradients
            optimizer.zero_grad()
            # 7) Accumulate loss and accuracy
            #    (Hint: Detach loss tensor before accumulation)
            #    (Hint: Move tensors to CPU for accumulation)
            loss_train += loss.detach().cpu()
            acc_train += acc.detach().cpu()

        loss_train = (loss_train/len(loader_train)).item()
        acc_train = (acc_train/len(loader_train)).item()
        # Report loss and metrics
        print(f"Ending training of epoch {i+1}/{n_epochs}: loss: {loss_train}, accuracy: {acc_train}")

        # Then evaluate the model
        loss_vali = 0.
        acc_vali = 0.

        # Put model in validation mode
        model.eval()

        print(f"Starting validation epoch {i+1}/{n_epochs} ...")
        for captcha_images_batch, texts_class_indices_batch in tqdm(loader_vali):
            ## [ TODO ]
            # Complete the validation inner loop:
            #
            # 1) Move images and labels to the target device
            # 2) Perform forward pass to get logits
            # 3) Compute loss using `multi_char_loss` and accuracy using `multi_char_acc`
            # 4) Accumulate loss and accuracy (move tensors to CPU for accumulation)
            #    (Hint: Move tensors to CPU for accumulation)
            captcha_images_batch = captcha_images_batch.to(device)
            texts_class_indices_batch = texts_class_indices_batch.to(device)
            output = model(captcha_images_batch)

            loss = multi_char_loss(output, texts_class_indices_batch)
            acc = multi_char_acc(output, texts_class_indices_batch)
            loss_vali += loss.detach().cpu()
            acc_vali += acc.detach().cpu()


        loss_vali = (loss_vali/len(loader_vali)).item()
        acc_vali = (acc_vali/len(loader_vali)).item()
        # Report loss and metrics
        print(f"Ending validation of epoch {i+1}/{n_epochs}: loss: {loss_vali}, accuracy: {acc_vali}")

        # Save epoch metrics
        history.append({
            "epoch": i+1,
            "loss_train": loss_train,
            "acc_train": acc_train,
            "loss_vali": loss_vali,
            "acc_vali": acc_vali
        })

        # Put model back in training mode
        model.train()

    return history
```

Ok, it's time to put everything together. We will build our model, train our model for some epochs and save it as `model-basic-params.pt`. For model training, you may adjust the number of epochs and / or batch size if necessary, to ensure that you achieve at least 80% of accuracy on the training set and at least 70% of accuracy on the validation set.

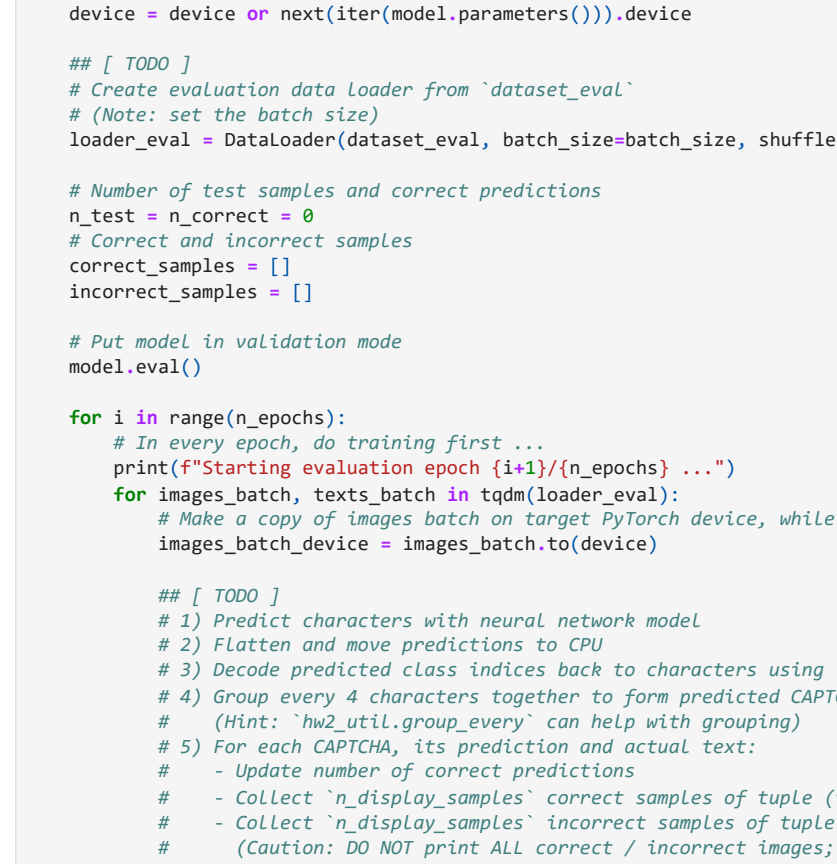
In [66]:
```
from torch.optim import AdamW

# Detect best PyTorch device
# (Prefer GPU devices such as CUDA or MPS, then fall back to CPU as last resort)
torch_device = hw2_util.get_torch_device()

# Build training and validation datasets
dataset_train = CAPTCHADataset(captcha_images_train, texts_class_indices_train, captcha_transform)
dataset_vali = CAPTCHADataset(captcha_images_vali, texts_class_indices_vali, captcha_transform)

# Build a basic end-to-end CAPTCHA model
model_basic = build_model(n_classes)
# Move model to PyTorch device
model_basic = model_basic.to(torch_device)
# Build an AdamW optimizer with model parameters
optimizer = AdamW(model_basic.parameters(), lr=0.001)

# Show model structure
print("Model structure:")
print(model_basic)
print()

# Train the basic model for 100 epochs
history_basic = train_model(model_basic, optimizer, dataset_train, dataset_vali)
# Save the basic model in file
torch.save(model_basic.state_dict(), "./model-basic-params.pt")
```

```
Model structure:
Sequential(
  (0): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (1): ReLU()
  (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (3): Conv2d(10, 40, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (5): Conv2d(40, 100, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (6): ReLU()
  (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (8): Conv2d(100, 800, kernel_size=(3, 3), stride=(1, 2))
  (9): Conv2d(800, 32, kernel_size=(1, 1), stride=(1, 1))
  (10): Conv2d(32, 32, kernel_size=(1, 1), stride=(1, 1))
  (11): Flatten(start_dim=2, end_dim=-1)
  (12): Transpose(dim0=1, dim1=2)
)

Starting training ...
Starting training epoch 1/50 ...
100%|██████████| 22/22 [00:00<00:00, 23.17it/s]
Ending training of epoch 1/50: loss: 3.394390344619751, accuracy: 0.0
Starting validation epoch 1/50 ...
100%|██████████| 8/8 [00:00<00:00, 39.50it/s]
Ending validation of epoch 1/50: loss: 3.2835230827331543, accuracy: 0.0
Starting training epoch 2/50 ...
100%|██████████| 22/22 [00:01<00:00, 13.22it/s]
Ending training of epoch 2/50: loss: 3.1547799110412598, accuracy: 0.0
Starting validation epoch 2/50 ...
100%|██████████| 8/8 [00:00<00:00, 13.96it/s]
Ending validation of epoch 2/50: loss: 2.9669809341430664, accuracy: 0.0
Starting training epoch 3/50 ...
100%|██████████| 22/22 [00:02<00:00,  7.92it/s]
Ending training of epoch 3/50: loss: 2.8647263050079346, accuracy: 0.0
Starting validation epoch 3/50 ...
100%|██████████| 8/8 [00:00<00:00, 34.18it/s]
Ending validation of epoch 3/50: loss: 2.687664747238159, accuracy: 0.0
Starting training epoch 4/50 ...
100%|██████████| 22/22 [00:01<00:00, 21.65it/s]
Ending training of epoch 4/50: loss: 2.628237009048462, accuracy: 0.0
Starting validation epoch 4/50 ...
100%|██████████| 8/8 [00:00<00:00, 40.01it/s]
Ending validation of epoch 4/50: loss: 2.4765796661376953, accuracy: 0.0
Starting training epoch 5/50 ...
100%|██████████| 22/22 [00:00<00:00, 22.55it/s]
Ending training of epoch 5/50: loss: 2.242919921875, accuracy: 0.008522727526724339
Starting validation epoch 5/50 ...
100%|██████████| 8/8 [00:00<00:00, 40.30it/s]
Ending validation of epoch 5/50: loss: 2.1338517665863037, accuracy: 0.0078125
Starting training epoch 6/50 ...
100%|██████████| 22/22 [00:01<00:00, 20.85it/s]
Ending training of epoch 6/50: loss: 1.8873157501220703, accuracy: 0.026988636702299118
Starting validation epoch 6/50 ...
100%|██████████| 8/8 [00:00<00:00, 41.12it/s]
Ending validation of epoch 6/50: loss: 1.5380407571792603, accuracy: 0.08984375
Starting training epoch 7/50 ...
100%|██████████| 22/22 [00:02<00:00, 10.56it/s]
Ending training of epoch 7/50: loss: 1.4196921586990356, accuracy: 0.12231692671775818
Starting validation epoch 7/50 ...
100%|██████████| 8/8 [00:00<00:00, 13.33it/s]
Ending validation of epoch 7/50: loss: 1.3902424573898315, accuracy: 0.1640625
Starting training epoch 8/50 ...
100%|██████████| 22/22 [00:02<00:00, 10.34it/s]
Ending training of epoch 8/50: loss: 1.1247559785842896, accuracy: 0.23832069337368011
Starting validation epoch 8/50 ...
100%|██████████| 8/8 [00:00<00:00, 36.93it/s]
Ending validation of epoch 8/50: loss: 1.2863141298294067, accuracy: 0.1484375
Starting training epoch 9/50 ...
100%|██████████| 22/22 [00:00<00:00, 23.23it/s]
Ending training of epoch 9/50: loss: 0.9795593023300171, accuracy: 0.32181188464164734
Starting validation epoch 9/50 ...
100%|██████████| 8/8 [00:00<00:00, 36.56it/s]
Ending validation of epoch 9/50: loss: 0.7760893106460571, accuracy: 0.4192708432674408
Starting training epoch 10/50 ...
100%|██████████| 22/22 [00:00<00:00, 22.46it/s]
Ending training of epoch 10/50: loss: 0.8420100808143616, accuracy: 0.4370265007019043
Starting validation epoch 10/50 ...
100%|██████████| 8/8 [00:00<00:00, 39.65it/s]
Ending validation of epoch 10/50: loss: 0.8090511560440063, accuracy: 0.4348958432674408
Starting training epoch 11/50 ...
100%|██████████| 22/22 [00:01<00:00, 21.68it/s]
Ending training of epoch 11/50: loss: 0.7017028331756592, accuracy: 0.48768940567970276
Starting validation epoch 11/50 ...
100%|██████████| 8/8 [00:00<00:00, 34.13it/s]
Ending validation of epoch 11/50: loss: 0.6956894993782043, accuracy: 0.5065104365348816
Starting training epoch 12/50 ...
100%|██████████| 22/22 [00:02<00:00,  9.44it/s]
Ending training of epoch 12/50: loss: 0.6598713397979736, accuracy: 0.5864899158477783
Starting validation epoch 12/50 ...
100%|██████████| 8/8 [00:00<00:00, 16.27it/s]
Ending validation of epoch 12/50: loss: 0.5197509527206421, accuracy: 0.66796875
Starting training epoch 13/50 ...
100%|██████████| 22/22 [00:01<00:00, 11.99it/s]
Ending training of epoch 13/50: loss: 0.5716153383255005, accuracy: 0.5997474789619446
Starting validation epoch 13/50 ...
100%|██████████| 8/8 [00:00<00:00, 40.09it/s]
Ending validation of epoch 13/50: loss: 0.5846482515335083, accuracy: 0.5690104365348816
Starting training epoch 14/50 ...
100%|██████████| 22/22 [00:01<00:00, 19.53it/s]
Ending training of epoch 14/50: loss: 0.5556275248527527, accuracy: 0.6526199579238892
Starting validation epoch 14/50 ...
100%|██████████| 8/8 [00:00<00:00, 37.43it/s]
Ending validation of epoch 14/50: loss: 0.5543498396873474, accuracy: 0.6575520634651184
Starting training epoch 15/50 ...
100%|██████████| 22/22 [00:01<00:00, 21.26it/s]
Ending training of epoch 15/50: loss: 0.4327791929244995, accuracy: 0.6980745196342468
Starting validation epoch 15/50 ...
100%|██████████| 8/8 [00:00<00:00, 43.91it/s]
Ending validation of epoch 15/50: loss: 0.5712497234344482, accuracy: 0.6419270634651184
Starting training epoch 16/50 ...
100%|██████████| 22/22 [00:01<00:00, 20.19it/s]
Ending training of epoch 16/50: loss: 0.4399925172328949, accuracy: 0.6908144354820251
Starting validation epoch 16/50 ...
100%|██████████| 8/8 [00:00<00:00, 15.76it/s]
Ending validation of epoch 16/50: loss: 0.7100648880004883, accuracy: 0.6158854365348816
Starting training epoch 17/50 ...
100%|██████████| 22/22 [00:02<00:00,  9.19it/s]
Ending training of epoch 17/50: loss: 0.4483382999897003, accuracy: 0.7107007503509521
Starting validation epoch 17/50 ...
100%|██████████| 8/8 [00:00<00:00, 15.12it/s]
Ending validation of epoch 17/50: loss: 0.34219473600387573, accuracy: 0.73046875
Starting training epoch 18/50 ...
100%|██████████| 22/22 [00:01<00:00, 15.77it/s]
Ending training of epoch 18/50: loss: 0.44521406292915344, accuracy: 0.7023358941078186
Starting validation epoch 18/50 ...
100%|██████████| 8/8 [00:00<00:00, 39.99it/s]
Ending validation of epoch 18/50: loss: 0.5528303384780884, accuracy: 0.6158854365348816
Starting training epoch 19/50 ...
100%|██████████| 22/22 [00:01<00:00, 21.43it/s]
Ending training of epoch 19/50: loss: 0.4694134593009949, accuracy: 0.7228535413742065
Starting validation epoch 19/50 ...
100%|██████████| 8/8 [00:00<00:00, 42.03it/s]
Ending validation of epoch 19/50: loss: 0.32945576310157776, accuracy: 0.7356770634651184
Starting training epoch 20/50 ...
100%|██████████| 22/22 [00:01<00:00, 21.51it/s]
Ending training of epoch 20/50: loss: 0.4153304398059845, accuracy: 0.7271149158477783
Starting validation epoch 20/50 ...
100%|██████████| 8/8 [00:00<00:00, 35.79it/s]
Ending validation of epoch 20/50: loss: 0.49718958139419556, accuracy: 0.6731770634651184
Starting training epoch 21/50 ...
100%|██████████| 22/22 [00:01<00:00, 15.65it/s]
Ending training of epoch 21/50: loss: 0.3638571798801422, accuracy: 0.7293245196342468
Starting validation epoch 21/50 ...
100%|██████████| 8/8 [00:00<00:00, 15.66it/s]
Ending validation of epoch 21/50: loss: 0.5062099099159241, accuracy: 0.66015625
Starting training epoch 22/50 ...
100%|██████████| 22/22 [00:02<00:00,  9.45it/s]
Ending training of epoch 22/50: loss: 0.35319381952285767, accuracy: 0.7640467286109924
Starting validation epoch 22/50 ...
100%|██████████| 8/8 [00:00<00:00, 16.50it/s]
Ending validation of epoch 22/50: loss: 0.41993075609207153, accuracy: 0.7265625
Starting training epoch 23/50 ...
100%|██████████| 22/22 [00:01<00:00, 18.48it/s]
Ending training of epoch 23/50: loss: 0.30212417244911194, accuracy: 0.7776199579238892
Starting validation epoch 23/50 ...
100%|██████████| 8/8 [00:00<00:00, 40.71it/s]
Ending validation of epoch 23/50: loss: 0.4303743243217468, accuracy: 0.6979166865348816
Starting training epoch 24/50 ...
100%|██████████| 22/22 [00:01<00:00, 21.19it/s]
Ending training of epoch 24/50: loss: 0.3450290858745575, accuracy: 0.7638888955116272
Starting validation epoch 24/50 ...
100%|██████████| 8/8 [00:00<00:00, 39.21it/s]
Ending validation of epoch 24/50: loss: 0.3827379643917084, accuracy: 0.7578125
Starting training epoch 25/50 ...
100%|██████████| 22/22 [00:01<00:00, 21.72it/s]
Ending training of epoch 25/50: loss: 0.30650314688682556, accuracy: 0.7810922265052795
Starting validation epoch 25/50 ...
100%|██████████| 8/8 [00:00<00:00, 38.70it/s]
Ending validation of epoch 25/50: loss: 0.2809964120388031, accuracy: 0.78515625
Starting training epoch 26/50 ...
100%|██████████| 22/22 [00:01<00:00, 12.70it/s]
Ending training of epoch 26/50: loss: 0.30803683400154114, accuracy: 0.7867740392684937
Starting validation epoch 26/50 ...
100%|██████████| 8/8 [00:00<00:00, 16.54it/s]
Ending validation of epoch 26/50: loss: 0.33412420749664307, accuracy: 0.7473958134651184
Starting training epoch 27/50 ...
100%|██████████| 22/22 [00:02<00:00,  8.61it/s]
Ending training of epoch 27/50: loss: 0.27418431639671326, accuracy: 0.8072916269302368
Starting validation epoch 27/50 ...
100%|██████████| 8/8 [00:00<00:00, 36.17it/s]
Ending validation of epoch 27/50: loss: 0.26578933000564575, accuracy: 0.8359375
Starting training epoch 28/50 ...
100%|██████████| 22/22 [00:01<00:00, 21.61it/s]
Ending training of epoch 28/50: loss: 0.3312202990055084, accuracy: 0.7916666269302368
Starting validation epoch 28/50 ...
100%|██████████| 8/8 [00:00<00:00, 37.00it/s]
Ending validation of epoch 28/50: loss: 0.29154446721076965, accuracy: 0.77734375
Starting training epoch 29/50 ...
100%|██████████| 22/22 [00:00<00:00, 22.43it/s]
Ending training of epoch 29/50: loss: 0.2832498252391815, accuracy: 0.8131313323974609
Starting validation epoch 29/50 ...
100%|██████████| 8/8 [00:00<00:00, 36.59it/s]
Ending validation of epoch 29/50: loss: 0.4266407787799835, accuracy: 0.6705729365348816
Starting training epoch 30/50 ...
100%|██████████| 22/22 [00:00<00:00, 22.22it/s]
Ending training of epoch 30/50: loss: 0.29023802280426025, accuracy: 0.798768937587738
Starting validation epoch 30/50 ...
100%|██████████| 8/8 [00:00<00:00, 36.21it/s]
Ending validation of epoch 30/50: loss: 0.2553746700286865, accuracy: 0.828125
Starting training epoch 31/50 ...
100%|██████████| 22/22 [00:01<00:00, 18.38it/s]
Ending training of epoch 31/50: loss: 0.2643391788005829, accuracy: 0.815025269985199
Starting validation epoch 31/50 ...
100%|██████████| 8/8 [00:00<00:00, 31.02it/s]
Ending validation of epoch 31/50: loss: 0.32316455245018005, accuracy: 0.77734375
Starting training epoch 32/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.86it/s]
Ending training of epoch 32/50: loss: 0.33650484681129456, accuracy: 0.7825126647949219
Starting validation epoch 32/50 ...
100%|██████████| 8/8 [00:00<00:00, 31.37it/s]
Ending validation of epoch 32/50: loss: 0.47325319051742554, accuracy: 0.7864583134651184
Starting training epoch 33/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.62it/s]
Ending training of epoch 33/50: loss: 0.3004509508609772, accuracy: 0.8046085834503174
Starting validation epoch 33/50 ...
100%|██████████| 8/8 [00:00<00:00, 25.68it/s]
Ending validation of epoch 33/50: loss: 0.5139180421829224, accuracy: 0.7591145634651184
Starting training epoch 34/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.33it/s]
Ending training of epoch 34/50: loss: 0.20321135222911835, accuracy: 0.8592172265052795
Starting validation epoch 34/50 ...
100%|██████████| 8/8 [00:00<00:00, 31.97it/s]
Ending validation of epoch 34/50: loss: 0.3813486397266388, accuracy: 0.78515625
Starting training epoch 35/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.90it/s]
Ending training of epoch 35/50: loss: 0.2563493549823761, accuracy: 0.8308081030845642
Starting validation epoch 35/50 ...
100%|██████████| 8/8 [00:00<00:00, 32.66it/s]
Ending validation of epoch 35/50: loss: 0.301395446062088, accuracy: 0.8125
Starting training epoch 36/50 ...
100%|██████████| 22/22 [00:01<00:00, 18.24it/s]
Ending training of epoch 36/50: loss: 0.27579328417778015, accuracy: 0.8166036009788513
Starting validation epoch 36/50 ...
100%|██████████| 8/8 [00:00<00:00, 38.38it/s]
Ending validation of epoch 36/50: loss: 0.26266050338745117, accuracy: 0.8515625
Starting training epoch 37/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.27it/s]
Ending training of epoch 37/50: loss: 0.2556898891925812, accuracy: 0.8044507503509521
Starting validation epoch 37/50 ...
100%|██████████| 8/8 [00:00<00:00, 29.98it/s]
Ending validation of epoch 37/50: loss: 0.40405163168907166, accuracy: 0.67578125
Starting training epoch 38/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.00it/s]
Ending training of epoch 38/50: loss: 0.24971288442611694, accuracy: 0.8125
Starting validation epoch 38/50 ...
100%|██████████| 8/8 [00:00<00:00, 34.05it/s]
Ending validation of epoch 38/50: loss: 0.4387941062450409, accuracy: 0.7669270634651184
Starting training epoch 39/50 ...
100%|██████████| 22/22 [00:01<00:00, 16.05it/s]
Ending training of epoch 39/50: loss: 0.2580265700817108, accuracy: 0.8172348141670227
Starting validation epoch 39/50 ...
100%|██████████| 8/8 [00:00<00:00, 28.70it/s]
Ending validation of epoch 39/50: loss: 0.3543247580528259, accuracy: 0.77734375
Starting training epoch 40/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.22it/s]
Ending training of epoch 40/50: loss: 0.2695735692977905, accuracy: 0.8358585834503174
Starting validation epoch 40/50 ...
100%|██████████| 8/8 [00:00<00:00, 32.18it/s]
Ending validation of epoch 40/50: loss: 0.22186154127120972, accuracy: 0.875
Starting training epoch 41/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.07it/s]
Ending training of epoch 41/50: loss: 0.19437995553016663, accuracy: 0.8677399158477783
Starting validation epoch 41/50 ...
100%|██████████| 8/8 [00:00<00:00, 31.35it/s]
Ending validation of epoch 41/50: loss: 0.36888784170150757, accuracy: 0.7981770634651184
Starting training epoch 42/50 ...
100%|██████████| 22/22 [00:01<00:00, 16.48it/s]
Ending training of epoch 42/50: loss: 0.24467416107654572, accuracy: 0.8355429172515869
Starting validation epoch 42/50 ...
100%|██████████| 8/8 [00:00<00:00, 33.59it/s]
Ending validation of epoch 42/50: loss: 0.2734644412994385, accuracy: 0.8098958134651184
Starting training epoch 43/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.80it/s]
Ending training of epoch 43/50: loss: 0.19053365290164948, accuracy: 0.8521149158477783
Starting validation epoch 43/50 ...
100%|██████████| 8/8 [00:00<00:00, 35.58it/s]
Ending validation of epoch 43/50: loss: 0.2820509970188141, accuracy: 0.85546875
Starting training epoch 44/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.64it/s]
Ending training of epoch 44/50: loss: 0.2059314101934433, accuracy: 0.8821022510528564
Starting validation epoch 44/50 ...
100%|██████████| 8/8 [00:00<00:00, 34.48it/s]
Ending validation of epoch 44/50: loss: 0.2804834842681885, accuracy: 0.7825520634651184
Starting training epoch 45/50 ...
100%|██████████| 22/22 [00:01<00:00, 16.88it/s]
Ending training of epoch 45/50: loss: 0.2405112385749817, accuracy: 0.8364899158477783
Starting validation epoch 45/50 ...
100%|██████████| 8/8 [00:00<00:00, 28.85it/s]
Ending validation of epoch 45/50: loss: 0.3049268424510956, accuracy: 0.7486979365348816
Starting training epoch 46/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.22it/s]
Ending training of epoch 46/50: loss: 0.2302754521369934, accuracy: 0.8352272510528564
Starting validation epoch 46/50 ...
100%|██████████| 8/8 [00:00<00:00, 23.92it/s]
Ending validation of epoch 46/50: loss: 0.31066805124282837, accuracy: 0.7552083134651184
Starting training epoch 47/50 ...
100%|██████████| 22/22 [00:01<00:00, 16.00it/s]
Ending training of epoch 47/50: loss: 0.21673573553562164, accuracy: 0.8301767706871033
Starting validation epoch 47/50 ...
100%|██████████| 8/8 [00:00<00:00, 35.02it/s]
Ending validation of epoch 47/50: loss: 0.25124621391296387, accuracy: 0.83984375
Starting training epoch 48/50 ...
100%|██████████| 22/22 [00:01<00:00, 15.77it/s]
Ending training of epoch 48/50: loss: 0.18896864354610443, accuracy: 0.8450126647949219
Starting validation epoch 48/50 ...
100%|██████████| 8/8 [00:00<00:00, 32.03it/s]
Ending validation of epoch 48/50: loss: 0.22448185086250305, accuracy: 0.7903645634651184
Starting training epoch 49/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.10it/s]
Ending training of epoch 49/50: loss: 0.18425193428993225, accuracy: 0.8533775210380554
Starting validation epoch 49/50 ...
100%|██████████| 8/8 [00:00<00:00, 34.23it/s]
Ending validation of epoch 49/50: loss: 0.24988123774528503, accuracy: 0.7981770634651184
Starting training epoch 50/50 ...
100%|██████████| 22/22 [00:01<00:00, 17.97it/s]
Ending training of epoch 50/50: loss: 0.24272726476192474, accuracy: 0.8429608941078186
Starting validation epoch 50/50 ...
100%|██████████| 8/8 [00:00<00:00, 28.61it/s]
Ending validation of epoch 50/50: loss: 0.252610981464386, accuracy: 0.7942708134651184
```

Evaluation

We have obtained a trained model as well as the training history during the previous stage. The training history contains the loss and accuracy of the nerual network as the training process proceeded. To show their trend during training, we plot these metrics for both training and validation set:

In [68]:
```
from matplotlib import pyplot as plt
import numpy as np

## [ TODO ]
# 1) Plot the loss of the model during training for both training set and validation set.
#    (Hint: for history list of dicts, training loss is `history[epoch]["loss_train"]` and `history[epoch]["loss_val
# 2) Plot the accuracy of the model during training for both training set and validation set.
#    (Hint: for history list of dicts, training accuracy is `history[epoch]["acc_train"]` and `history[epoch]["acc_v
#    (Note: plot all curves for the same metric on the same graph for comparison)
x = np.arange(0,50)
loss_train = [h["loss_train"] for h in history_basic]
loss_vali = [h["loss_vali"] for h in history_basic]
acc_train= [h["acc_train"] for h in history_basic]
acc_vali = [h["acc_vali"] for h in history_basic]
plt.plot(x, loss_train, label = 'training_loss')
plt.plot(x, loss_vali, label = 'valid_loss')
plt.plot(x, acc_train, label = 'train_acc')
plt.plot(x, acc_vali, label = 'valid_acc')
plt.legend()
plt.show()
```



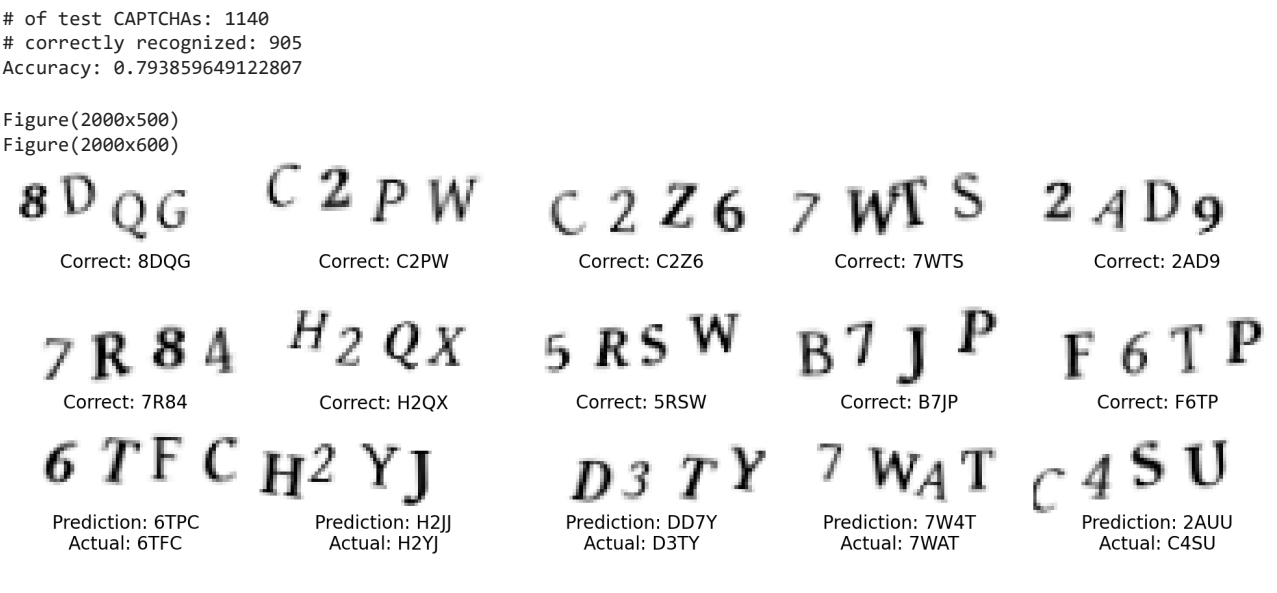
Next, we evaluate our model end-to-end on the test set. Like training and validation set, we transform test set CAPTCHA images through data augmentation. We then predict the characters for these images and compare the predictions with ground truth. Similar to lab 3, we compute the accuracy of our model and show samples of correct and incorrect predictions:

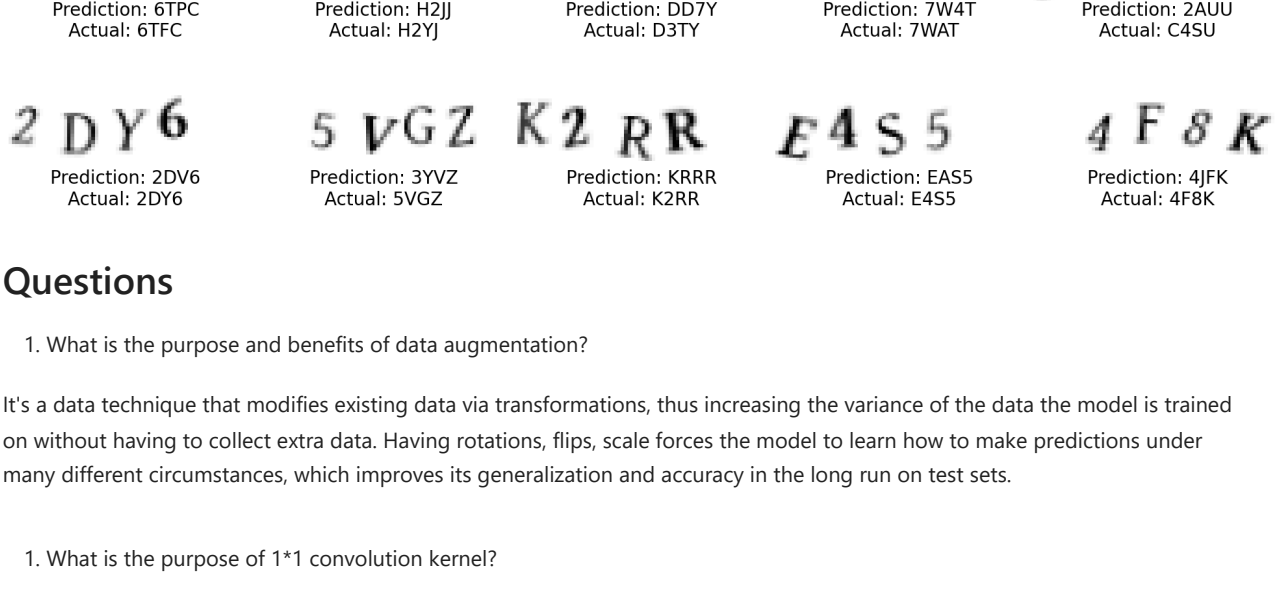
In [87]:
```
def evaluate_model(model, le, dataset_eval, batch_size=32, n_epochs=5, n_display_samples=10, device=None):
    # Detect device from model parameters if not provided
    device = device or next(iter(model.parameters())).device

    ## [ TODO ]
    # Create evaluation data loader from `dataset_eval`
    # (Note: set the batch size)
    loader_eval = DataLoader(dataset_eval, batch_size=batch_size, shuffle=False)

    # Number of test samples and correct predictions
    n_test = n_correct = 0
    # Correct and incorrect samples
    correct_samples = []
    incorrect_samples = []

    # Put model in validation mode
    model.eval()

    for i in range(n_epochs):
        # In every epoch, do training first ...
        print(f"Starting evaluation epoch {i+1}/{n_epochs} ...")
        for images_batch, texts_batch in tqdm(loader_eval):
            # Make a copy of images batch on target PyTorch device, while keeping original batch on CPU
            images_batch_device = images_batch.to(device)

            ## [ TODO ]
            # 1) Predict characters with neural network model
            # 2) Flatten and move predictions to CPU
            # 3) Decode predicted class indices back to characters using `LabelEncoder.inverse_transform`
            # 4) Group every 4 characters together to form predicted CAPTCHAs
            #    (Hint: `hw2_util.group_every` can help with grouping)
            # 5) For each CAPTCHA, its prediction and actual text:
            #    - Update number of correct predictions
            #    - Collect `n_display_samples` correct samples of tuple (image, prediction) for review
            #    - Collect `n_display_samples` incorrect samples of tuple (image, prediction, actual) for review
            #      (Caution: DO NOT print ALL correct / incorrect images; this is slow and the output will be messy
            pred = model(images_batch_device)
            pred_classes = pred.argmax(dim=2)
            pred_classes = pred_classes.detach().cpu().numpy().flatten()
            chars = le.inverse_transform(pred_classes)
            char_group = hw2_util.group_every(chars, 4)
            n_test += len(texts_batch)
            for i, (captcha, text) in enumerate(zip(char_group, texts_batch)):
                captcha = "".join(captcha)
                img = images_batch[i].squeeze(0).cpu().numpy()
                if (captcha == text):
                    n_correct += 1
                    if len(correct_samples) < n_display_samples:
                        correct_samples.append((img, captcha))
                else:
                    if len(incorrect_samples) < n_display_samples:
                        incorrect_samples.append((img, captcha, text))

        # Report loss and metrics
        print(f"Ending evaluation of epoch {i+1}/{n_epochs} ...")

    # Put model back in training mode
    model.train()

    # Show statistics
    print()
    print("# of test CAPTCHAs:", n_test)
    print("# correctly recognized:", n_correct)
    print("Accuracy:", n_correct/n_test, "\n")

    assert len(correct_samples)<=n_display_samples, \
        f"you should only display {n_display_samples} correct samples as examples!"
    assert len(incorrect_samples)<=n_display_samples, \
        f"you should only display {n_display_samples} incorrect samples as examples!"

    # Show all correct predictions
    hw2_util.print_images(
        hw2_util.unzip(correct_samples)[0],
        texts=[f"Correct: {actual}" for _, actual in correct_samples],
        n_rows=2
    )

    # Show all incorrect predictions
    hw2_util.print_images(
        hw2_util.unzip(incorrect_samples)[0],
        texts=[
            f"Prediction: {pred}\nActual: {actual}" \
            for _, pred, actual in incorrect_samples
        ],
        n_rows=2,
        fig_size=(20, 6),
        text_center=(0.5, -0.25)
    )
```

Now we run the evaluation process on the basic model:

In [88]:
```
# Build evaluation dataset
dataset_eval = CAPTCHADataset(captcha_images_test, captcha_texts_test, captcha_transform)

# Evaluate the basic model
evaluate_model(model_basic, le, dataset_eval)
```

```
Starting evaluation epoch 1/5 ...
100%|██████████| 8/8 [00:00<00:00, 43.72it/s]
Ending evaluation of epoch 4/5 ...
Starting evaluation epoch 2/5 ...
100%|██████████| 8/8 [00:00<00:00, 49.48it/s]
Ending evaluation of epoch 4/5 ...
Starting evaluation epoch 3/5 ...
100%|██████████| 8/8 [00:00<00:00, 42.82it/s]
Ending evaluation of epoch 4/5 ...
Starting evaluation epoch 4/5 ...
100%|██████████| 8/8 [00:00<00:00, 44.39it/s]
Ending evaluation of epoch 4/5 ...
Starting evaluation epoch 5/5 ...
100%|██████████| 8/8 [00:00<00:00, 42.67it/s]
Ending evaluation of epoch 4/5 ...

# of test CAPTCHAs: 1140
# correctly recognized: 905
Accuracy: 0.793859649122807

Figure(2000x500)
Figure(2000x600)
```





Questions

1. What is the purpose and benefits of data augmentation?

It's a data technique that modifies existing data via transformations, thus increasing the variance of the data the model is trained on without having to collect extra data. Having rotations, flips, scale forces the model to learn how to make predictions under many different circumstances, which improves its generalization and accuracy in the long run on test sets.

1. What is the purpose of 1*1 convolution kernel?

This kernel performs dimensionality reduction where the model compresses the information and decreases the parameters and computation for the next layer.

1. What is the purpose of dropout? Does it work on improving the convergence of training or on generalization? Give some reasons to support your idea.

Dropout is when you randomly select some neurons during training. This is a measure against overfitting, which could potentially improve its generalization. Dropping some nodes means that it is losing information that is trained on the specific training set, which makes the model more resilient to unseen data.

1. What is the purpose of batch normalization? Does it work on improving the convergence of training or on generalization? Give some reasons to support your idea.

Batch normalization normalizes inputs to each layer at every mini batch, which improves the convergence of training. This is because it keeps the distribution consistent, allowing for faster and more stable training in future layers by reducing the initial adaptation to previous layers.

Improvements (Optional; Not Graded)

Like homework 1, **you can optionally choose to work on two improvements to the neural network model**, and see if these modifications help with the accuracy. Please note that we won't grade your solutions for these questions, but we may give you feedbacks should you choose to complete them. We also encourage you to **reuse previous code and functions** as much as possible, and avoid repeating identicial code from previous part of this homework.

Neural network models are susceptible to over-fitting problems due to their huge amount of free parameters. When neural network model overfits, it performs extremely well on the training set, but can easily fail when new samples are added. There are three possible approaches to mitigate the over-fitting issue:

- Adding regularization (L1 and/or L2) to parameters of each linear or convolution layer.
- Adding dropout layers (`torch.nn.Dropout`) to the model.
- Adding batch normalization layers (`torch.nn.BatchNorm2d`) to the model.

Try one or a few of these approaches and compare their performance with the original model. Complete the following tasks by adding code and text cells above and compare their performance with the original model. Complete the following tasks by adding code and text cells below:

- Specify your choice of approaches and the details for each approach:
  - For regularization, this is the type of regularization and regularization factor for each convolution and linear layer.
  - For dropout layers, specify the number, position and dropout rate of all dropout layers.
  - For batch normalization layers, specify the number and position of all batch normalization layers.
- Plot the loss and accuracy for both training and validation set. Think about what curves to put on the same plot.
  - One idea is to put the same metric of all four models (basic model and three improvement models) for the same set (e.g. training set) on the same plot. In this case, you will get four plots in the end.
  - You don't need to include plots as images in the notebook. Instead, just provide the code that can plot these curves.
- Finally, report the accuracy of all three improvement models on the test set.

Submission

When you're done, push this completed notebook (including all code and discussion answers) to your GitHub repo, then submit the repo link on the Canvas submission page.

References

1. PyTorch API reference: https://pytorch.org/docs/stable/index.html
2. PyTorch Neural Network Modules: https://pytorch.org/docs/stable/nn.html
3. Over-fitting: https://en.wikipedia.org/wiki/Overfitting
4. Residual neural network: https://en.wikipedia.org/wiki/Residual_neural_network
5. Residual blocks - Building blocks of ResNet: https://towardsdatascience.com/residual-blocks-building-blocks-of-resnet-fd90ca15d6ec